

Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу

«Операционные системы»

Группа: М8О-209БВ-25

Студент: Гурьев Т.А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 24.09.26

Москва, 2026

Постановка задачи

Вариант 10.

Родительский процесс создаёт дочерний процесс (разные программы). Пользователь вводит имя файла, родитель открывает его на чтение и подменяет им `stdin` дочернего процесса; `stdout` дочернего процесса перенаправлен в `pipe1`, откуда родитель читает и выводит на экран.

В файле записаны команды вида «число<newline>». Дочерний процесс производит проверку этого числа на простоту. Если число составное, то дочерний процесс пишет это число в стандартный поток вывода. Если число отрицательное или простое, то тогда дочерний и родительский процессы завершаются. Количество чисел может быть произвольным. Дополнительное ограничение: из стандартной библиотеки языка Си запрещено использовать `stdio.h`, `threads.h` и `stdatomic.h`.

Общий метод и алгоритм решения

- `fork()` – создаёт дочерний процесс.
- `pipe(pipefd)` – создаёт канал для передачи данных между процессами.
- `dup2(oldfd, newfd)` – перенаправляет `stdin/stdout` на файл или канал.
- `open()` – открывает файл, `close()` – закрывает дескриптор.
- `read()/write()` – чтение и запись через файловый дескриптор.
- `execv()` – заменяет образ процесса другой программой.
- `waitpid()` – ожидает завершения дочернего процесса.

Parent открывает файл и создаёт `pipe1`, затем вызывает `fork()`. В ветке ребёнка `stdin` переопределяется на файл, `stdout` — на write-конец `pipe1` (оба через `dup2`), лишние дескрипторы закрываются, и вызывается `execv("./child", argv)` — процесс становится отдельной программой `child`, которая построчно читает числа и проверяет их на простоту: составные пишет в `stdout`, на простом/отрицательном завершается `_exit()`.

В ветке родителя закрываются файл и write-конец `pipe1`, после чего `parent` в цикле читает read-конец `pipe1` и пишет прочитанное в свой `stdout`; при завершении `child` все write-концы закрываются, `read()` возвращает EOF, и `parent` вызывает `waitpid()`. Все системные вызовы проверяются на ошибку через общую функцию `die_errno()`. Построчное чтение без `stdio.h` реализовано вручную в `read_line()` (`common.c`) — она буферизует данные между вызовами `read()`, так как один `read()` не гарантирует ровно одну строку.

Отдельный случай — числа 0 и 1: они не являются ни простыми, ни составными математически, и условие задачи явно не описывает такой сценарий. По уточнению преподавателя `child` отправляет через `pipe1` отдельное сообщение об ошибке ("ERROR: N is neither prime nor composite") и ПРОДОЛЖАЕТ обработку оставшихся чисел файла, не завершаясь — в отличие от простого/отрицательного числа.

Код программы

common.h

```
#ifndef COMMON_H
#define COMMON_H

#include <stddef.h>
#include <sys/types.h>

typedef struct {
    int fd;
    char buf[4096];
    size_t len;
    size_t pos;
} line_reader_t;

ssize_t read_line(line_reader_t *r, char *out, size_t out_size);

void line_reader_init(line_reader_t *r, int fd);

void die_errno(const char *msg);

#endif
```

common.c

```
#include "common.h"
#include <string.h>
#include <unistd.h>
#include <errno.h>

ssize_t read_line(line_reader_t *r, char *out, size_t out_size) {
    for (;;) {
        char *newline = memchr(r->buf + r->pos, '\n', r->len - r->pos);

        if (newline != NULL) {
            size_t line_len = newline - (r->buf + r->pos);
            if (out_size < line_len + 1) {
                errno = EOVERFLOW;
                return -1;
            }
            memcpy(out, r->buf + r->pos, line_len);
            out[line_len] = '\0';
            r->pos += line_len + 1;
            return line_len;
        } else {
            size_t remaining = r->len - r->pos;
            memmove(r->buf, r->buf + r->pos, remaining);
            r->pos = 0;
            r->len = remaining;
            if (r->len == sizeof(r->buf)) {
                errno = EOVERFLOW;
                return -1;
            }
            ssize_t n = read(r->fd, r->buf + r->len, sizeof(r->buf) - r->len);
            if (n < 0) {
                return -1;
            }
            if (n == 0) {
                if (r->len == 0) {
                    return 0;
                } else {
                    if (out_size < r->len + 1) {
                        errno = EOVERFLOW;
                        return -1;
                    }
                }
            }
        }
    }
}
```

```

        memcpy(out, r->buf, r->len);
        out[r->len] = '\0';
        size_t res = r->len;
        r->len = 0;
        return res;
    }
}
if (n > 0) {
    r->len += n;
}
}
}
}

void line_reader_init(line_reader_t *r, int fd) {
    r->fd = fd;
    r->len = 0;
    r->pos = 0;
}

void die_errno(const char *msg) {
    write(STDERR_FILENO, msg, strlen(msg));
    write(STDERR_FILENO, ": ", 2);
    const char *err_text = strerror(errno);
    write(STDERR_FILENO, err_text, strlen(err_text));
    write(STDERR_FILENO, "\n", 1);
    _exit(1);
}

```

child.c

```

#include <unistd.h>
#include <stdlib.h>
#include "common.h"

int is_prime(long n);
long parse_long(const char *str);
int write_long_line(int fd, long n);
void write_error_line(int fd, long n);

int main(void) {
    line_reader_t r;
    line_reader_init(&r, STDIN_FILENO);

    char line[256];

    for (;;) {
        ssize_t res = read_line(&r, line, sizeof(line));

        if (res < 0) {
            die_errno("child: ошибка чтения строки");
        }

        if (res == 0) {
            return 0;
        }
        long number = parse_long(line);

        if (number == 0 || number == 1) {
            write_error_line(STDOUT_FILENO, number);
            continue;
        }

        if (number < 0 || is_prime(number)) {
            _exit(0);
        }
        write_long_line(STDOUT_FILENO, number);
    }
}

```

```

int is_prime(long n) {
    if (n < 2) return 0;
    if (n == 2) return 1;
    if (n % 2 == 0) return 0;
    for (long i = 3; i * i <= n; i += 2) {
        if (n % i == 0) return 0;
    }
    return 1;
}

long parse_long(const char *str) {
    return strtol(str, NULL, 10);
}

int write_long_line(int fd, long n) {
    char buf[32];
    int pos = sizeof(buf);
    --pos;
    buf[pos] = '\n';
    if (n == 0) buf[--pos] = '0';
    while (n != 0) {
        long a = n % 10;
        buf[--pos] = '0' + a;
        n /= 10;
    }
    write(fd, buf + pos, sizeof(buf) - pos);
    return 0;
}

void write_error_line(int fd, long n) {
    char num_buf[32];
    int pos = sizeof(num_buf);
    if (n == 0) {
        num_buf[--pos] = '0';
    } else {
        long tmp = n;
        while (tmp != 0) {
            num_buf[--pos] = '0' + (tmp % 10);
            tmp /= 10;
        }
    }
    const char prefix[] = "ERROR: ";
    const char suffix[] = " is neither prime nor composite\n";
    write(fd, prefix, sizeof(prefix) - 1);
    write(fd, num_buf + pos, sizeof(num_buf) - pos);
    write(fd, suffix, sizeof(suffix) - 1);
}

```

parent.c

```

#include "common.h"
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
#include <sys/types.h>

int main(void) {
    line_reader_t r;
    line_reader_init(&r, STDIN_FILENO);
    char filename[256];
    ssize_t res = read_line(&r, filename, sizeof(filename));

    if (res < 0) {
        die_errno("parent: ошибка чтения строки");
    }
    if (res == 0) {
        const char msg[] = "parent: не введено имя файла\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        _exit(1);
    }
}

```

```

int fd_file = open(filename, O_RDONLY);
if (fd_file < 0) {
    die_errno("parent: open");
}

int pipe1[2];
if (pipe(pipe1) < 0) {
    die_errno("parent: pipe");
}

pid_t pid = fork();
if (pid < 0) {
    die_errno("parent: fork");
}
if (pid == 0) {
    if (dup2(fd_file, STDIN_FILENO) < 0) {
        die_errno("child: dup2 stdin");
    }
    if (dup2(pipe1[1], STDOUT_FILENO) < 0) {
        die_errno("child: dup2 stdout");
    }
    close(fd_file);
    close(pipe1[0]);
    close(pipe1[1]);
    char *argv[] = { "./child", NULL };
    execv("./child", argv);
    die_errno("child: execv");
}
close(fd_file);
close(pipe1[1]);
char buf[4096];
ssize_t n;
while ((n = read(pipe1[0], buf, sizeof(buf))) > 0) {
    write(STDOUT_FILENO, buf, n);
}
if (n < 0) {
    die_errno("parent: read");
}
close(pipe1[0]);
int status;
waitpid(pid, &status, 0);
return 0;
}

```

Протокол работы программы

Тестирование:

```

$ echo "test1.txt" | ./parent      # 4 6 8 9 10 – все составные
4
6
8
9
10

$ echo "test2.txt" | ./parent      # 4 6 17 100 – стоп на простом 17
4
6

$ echo "test3.txt" | ./parent      # 10 15 -3 20 – стоп на отрицательном -3
10
15

$ echo "test4.txt" | ./parent      # 12 9 17 без завершающего \n
12
9

$ echo "test5.txt" | ./parent      # 0 1 4 6 – ERROR на 0 и 1, обработка продолжается

```

```
ERROR: 0 is neither prime nor composite
ERROR: 1 is neither prime nor composite
4
6
```

```
$ echo "test6.txt" | ./parent      # 0 4 1 6 17 9 – ERROR вперемешку, стоп на 17
ERROR: 0 is neither prime nor composite
4
ERROR: 1 is neither prime nor composite
6
```

```
$ echo "no_such_file.txt" | ./parent
parent: open: No such file or directory
```

```
Проверка, что parent и child – разные процессы:
$ md5sum parent child
1d215e2b6b0a29ae4efc66ab12a5444f  parent
546ae62466f8724b6c06cae7f0231555  child
```

Вывод

Получены навыки управления процессами (fork, exec) и организации IPC через pipe с перенаправлением потоков (dup2). Основная сложность — запрет stdio.h: пришлось вручную реализовать построчное чтение поверх read() и вывод чисел без printf(). Отдельно важно закрытие неиспользуемых концов pipe — иначе read() у родителя не получает EOF. Отдельное уточнение потребовалось для чисел 0 и 1: они трактуются как отдельный тип ошибки (сообщение через pipe1) без остановки обработки остальных чисел файла.